Architecting a POSIX-Compliant Runtime Environment on ESP32

A Unikernel Approach to Dynamic ELF Execution

Dhruv

Department of Computer Science

June 20, 2026

Outline

The Problem: Static vs. Dynamic

- **Embedded Systems (RTOS):**
  - Highly efficient, deterministic.
  - **Static:** Monolithic firmware. Adding a feature requires recompiling and reflashing the entire OS.
  - "OTA Updates" replace the whole image.
- **General Purpose OS (Linux):**
  - Dynamic. Applications are separate binaries.
  - Can load/unload code at runtime ('exec').
  - Requires **MMU** (Memory Management Unit) and significant RAM.
- **The Gap:** Modern IoT (Edge Computing, C2, Drones) needs the dynamism of Linux on the hardware constraints of an MCU.

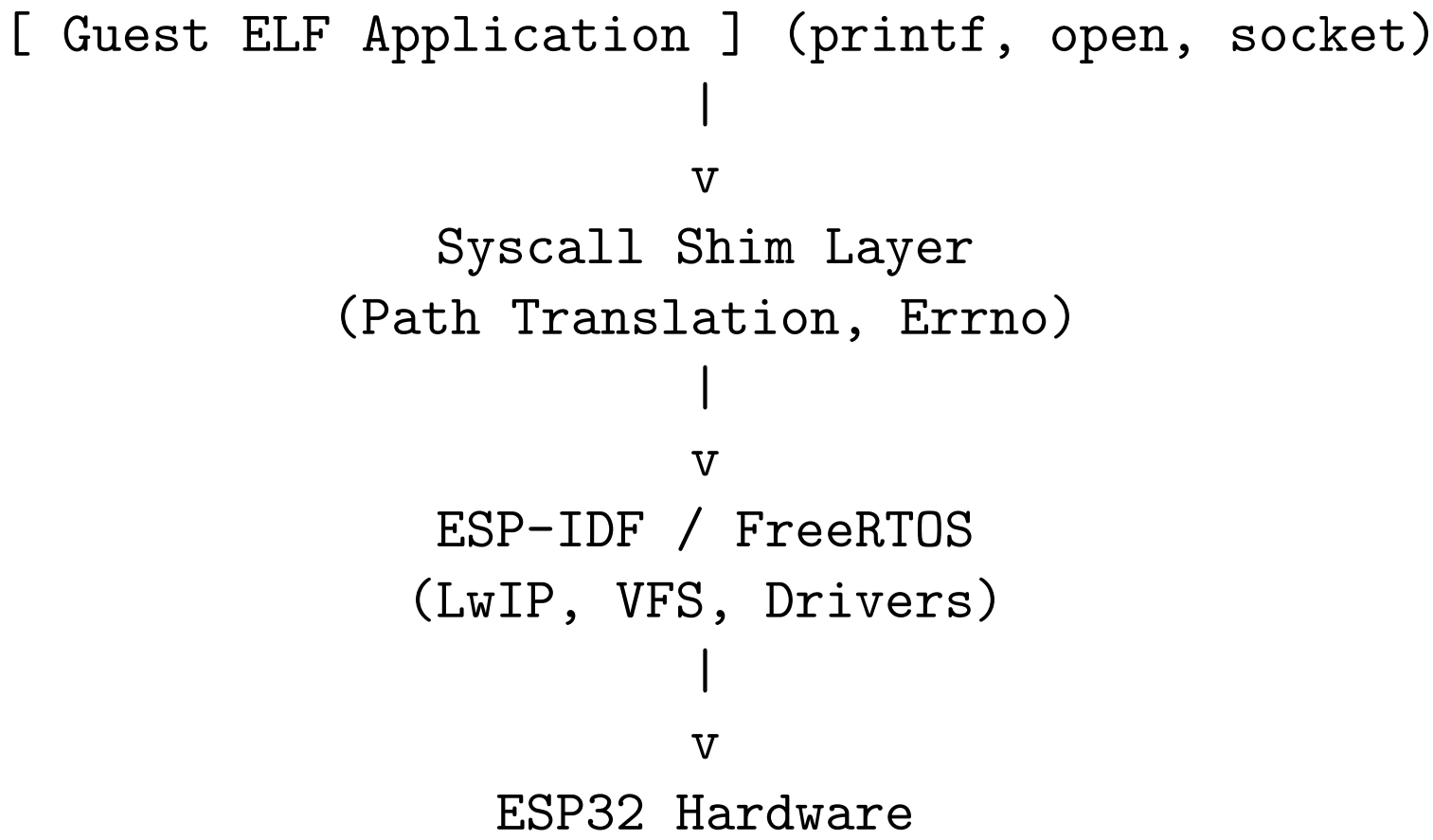
The Solution: A Thin Linux Compatibility Layer

Architecture: The Unikernel Model

Instead of running a full kernel, we implement a **Library OS** that translates Linux System Calls into ESP-IDF primitives.

- **No-MMU Constraint:** We cannot implement ‘fork()‘ or virtual memory.
- **Single Address Space:** The Kernel and Guest App share the same physical RAM.
- **ELF Loader:** A custom loader parses standard ‘.elf‘ binaries, allocates IRAM (Instruction RAM) for code, and performs *Xtensa-specific relocations* at load time.

System Architecture



Demo 1: Distributed Command Control

Scenario

An "Attacker" sends a compiled binary payload to an edge node (ESP32) over the network. The node executes it immediately and streams output back.

Workflow:

1. Master sends ELF Header + Data via TCP.
2. ESP32 saves to '/linux/payload.elf'.
3. **Redirection Shim:** 'dup2(socket, stdout)' allows 'printf' to stream over WiFi.
4. 'execve()' spawns a new task.

Key Tech:

- **shim_write:** Intercepts FD 1 (stdout) and dual-routes data to UART and TCP.
- **Spawn Model:** Simulates 'fork+exec' using 'xTaskCreate'.

Demo 2: Cyber-Physical System (V2X)

Scenario

An Edge-OS Collision Avoidance System. A Linux app performs real-time vector math to detect crashes and triggers hardware alarms.

- **Telemetry Bridge:** Web Dashboard sends JSON (GPS, Speed) $\rightarrow$ ESP32 $\rightarrow$ UDP $\rightarrow$ Linux App.
- **Vector Math (User Space):**

$$\vec{V}_{rel} = \vec{V}_{target} - \vec{V}_{ego}$$

$$TTC = \frac{|\vec{P}_{target} - \vec{P}_{ego}|}{|\vec{V}_{rel}|}$$

- **Hardware Abstraction (Kernel Space):**
  - App writes to '/dev/buzzer'.
- Custom VFS Driver translates 'write()' to 'ledc$_s et_d uty$()'(PWM).

Conclusion Future Work

- **Achievement:** Successfully implemented a POSIX-compliant runtime on ESP32 capable of running standard C applications.
- **Impact:** Enables "Write Once, Run Anywhere" for IoT edge logic without firmware reflashing.
- **Future Work:**
  - Implement ‘setjmp‘/‘longjmp‘ for fuller signal handling.
  - Add support for dynamic shared libraries (‘.so‘).
  - Port a lightweight scripting language (Lua/MicroPython) as a guest app.